

12. Mapper algoritms. Faktoru analīze

Jekaterīna Butkeviča

2026. gads 05. janvāris

Uzdevums

Atrodiet internetā kādu interesantu datu kopu, kurai varētu padarbināt: - Mapper algoritmu
- Faktoru analīzi

Dati

Tā kā uzdevumā bija teikts “Varat turpināt MD10 iesākto”, es mēģināju turpināt darbu ar unbalanced datiem, lai vieglāk uztvertu rezultātu atšķirības. Tomēr vienmēr saņēmu kļūdu, ka filtrs un intervālu skaits nav saskaņoti ar datu izkliedi. Tāpēc es nolēmu izmantot vīna datus, kas satur informāciju par dažādiem vīna ķīmiskajiem aspektiem, iegūtus, analizējot 178 vīna paraugus.

Papildus es normalizēju datus.

```
library(foreign)
vins <- read.arff("vins.arff")
vins_labels <- as.factor(vins$Type)
vins_numeric <- vins[,2:14]
vins_numeric <- scale(vins_numeric)
```

Mapper algoritms

Mapper algoritms ir topoloģiskas datu analīzes metode, ko izmanto, lai iegūtu priekšstatu par datu formu pat ļoti augstdimensionālos datos. Algoritms projicē augstdimensionālos datus caur izvēlētu lēcu, sadala tos pārklājošos intervālos, klasterizē katru apakškopu un savieno klasterus grafā, ja tiem ir kopīgi punkti. Rezultātā iegūst grafisku reprezentāciju, kas atklāj datu topoloģisko struktūru (zarus, pārejas, savienojumus), ko bieži nav iespējams saskatīt ar klasiskām metodēm, piemēram, PCA.

```
library(TDAmapper)
library(igraph)
library(ggplot2)
```

Kā lēcu (lens) es izvēlējos izmantot galveno komponentu analīzi (PCA). PCA metode ir visbiežāk izmantotā lēcas funkcija (Madukpe et al., 2025) un, iespējams, šāda izvēle

saistīta ar to, ka tā palīdz atklāt dabiskas grupas un gradientus, vienlaikus samazinot datu troksni.

Definēju lēcu:

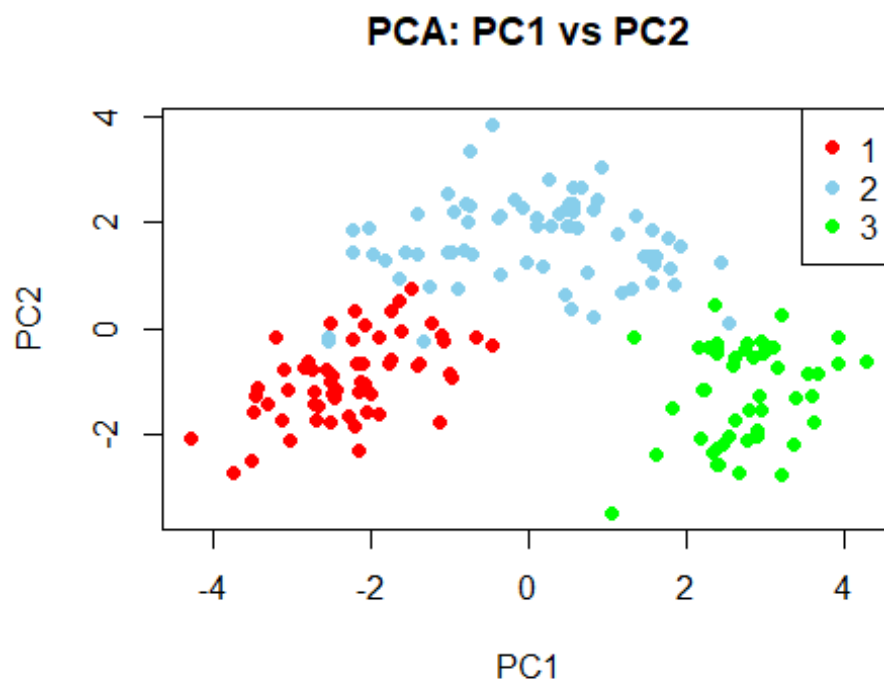
```
pca_res <- prcomp(vins_numeric)
filter_values <- pca_res$x[,1]
scores <- pca_res$x
```

Lai iegūtu priekšstatu par datu punktu izvietojumu pēc PCA, vizualizēju pirmās divas galvenās komponentes viena pret otru.

```
cols <- c("1" = "red", "2" = "skyblue", "3" = "green")
```

```
plot(scores[,1], scores[,2],
      col = cols[vins_labels],
      pch = 19,
      xlab = "PC1",
      ylab = "PC2",
      main = "PCA: PC1 vs PC2")
```

```
legend("topright",
      legend = levels(vins_labels),
      col = cols,
      pch = 19)
```



Attēlā redzams, ka, vizualizējot pirmās divas galvenās komponentes viena pret otru, iegūts izstiepts datu mākonis, kur trīs vīna šķirnes izvietojušās secīgi blakus viena otrai, ar minimālu savstarpēju pārklāšanos.

Definēju vienotu funkciju, lai kodu nevajadzētu atkārtot vairākas reizes:

```
run_mapper <- function(num_intervals, percent_overlap) {  
  mapper_obj <- mapper2D(  
    distance_matrix = dist(vins_numeric),  
    filter_values = list(filter_values, filter_values), # 2D filters  
    num_intervals = c(num_intervals, num_intervals),  
    percent_overlap = percent_overlap,  
    num_bins_when_clustering = 10  
  )  
  return(mapper_obj)  
}
```

Ekspperimentēju, kā jau ierasts, ar trīs dažādām vērtībām.

```
map_res1 <- run_mapper(3, 85)  
map_res2 <- run_mapper(5, 85)  
map_res3 <- run_mapper(7, 85)
```

Es mēģināju daudz dažādu variantu, bet ar mazāku pārklāšanos vai lielāku intervālu man visu laiku parādījās kļūda.

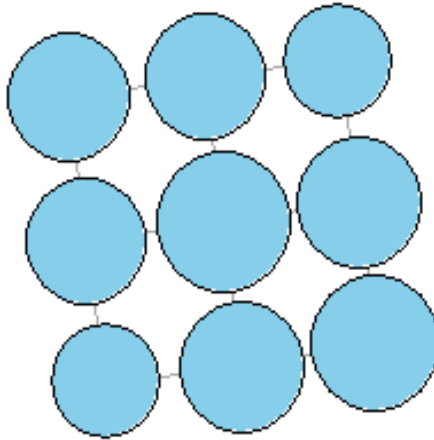
Definēju vizualizācijas funkciju, kurā krāsa tiek piešķirta pēc dominējošās vīna šķirnes.

```
plot_mapper_final <- function(m_obj, title) {  
  g <- graph.adjacency(m_obj$adjacency, mode="undirected")  
  
  node_class <- sapply(m_obj$points_in_vertex, function(pts) {  
    if (length(pts) == 0) return(NA)  
    names(which.max(table(vins_labels[pts])))  
  })  
  
  node_cols <- class_cols[node_class]  
  node_cols[is.na(node_cols)] <- "grey80"  
  
  plot(g, layout=layout_with_kk,  
        vertex.size=sqrt(sapply(m_obj$points_in_vertex, length))*7,  
        vertex.color=node_cols,  
        vertex.label=NA,  
        main=title)  
}  
class_cols <- c("1" = "red", "2" = "skyblue", "3" = "green")
```

Tālāk attēloju rezultātus:

```
plot_mapper_final(map_res1, "Mapper (n = 3, overlap = 85%)")
```

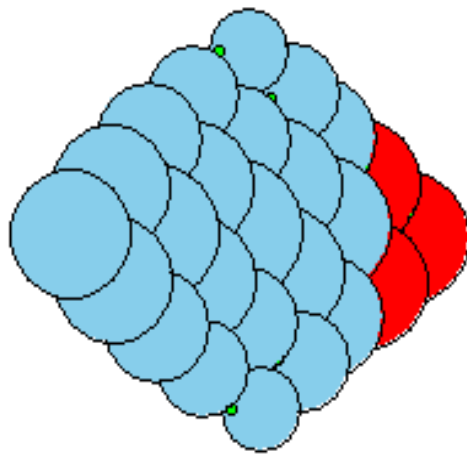
Mapper (n = 3, overlap = 85%)



Palaižot algoritmu ar 3 intervāliem un pārklāšanos 85%, nekas derīgs nav iegūts. Attēla redzams viena 3x3 grafa virsotņu “peļķe”. Virsotnes sturos nedaudz atšķiras izmērā. Es nolēmu, ka pareizāk grafa virsotni ir iekrāsot pēc dominejošas vīna šķirnes. Šajā attēlā visas virsotnes ir iekrastas zila krasā, kas atbilst otrai vīna šķirnei. Iespējams, šāds rezultāts ir iegūts tādēļ, kā otrā šķirnei pieder visvairāk punktu - 71 pret 59 punktiem pirmajai un 48 punktiem trešajai šķirnei.

```
plot_mapper_final(map_res2, "Mapper (n = 5, overlap = 85%)")
```

Mapper (n = 5, overlap = 85%)

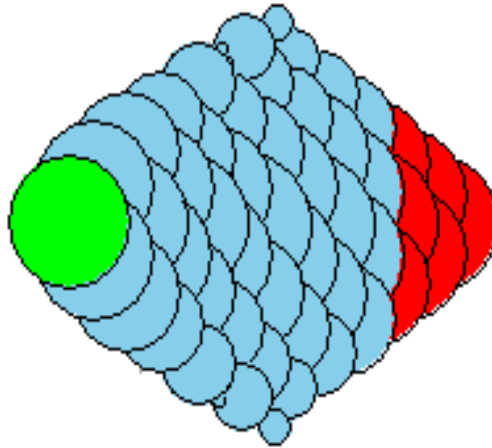


Palielinot intervālu skaitu līdz 5, attēlā parādās otrā krāsa – trīs virsotnes sāk reprezentēt pirmo vīna šķirni. Iespējams, tas notiek tāpēc, ka “1” šķirne ir otrā pēc novērojumu skaita (“1” n = 59, “2” n = 71, “3” n = 49). Zilo punktu izmēri nedaudz atšķiras – visticamāk, punktu blīvums bija mazāks tajos apgabalos, kur parādījās mazākie punkti. Grafiks kopumā izskatās kā priežu čiekurs, taču arī datu mākonis pēc PCA pielietošanas bija samērā vienvēidīgs, bez izteiktām slēptām formu niansēm.

Vēlāk, ieskatoties uzmanīgāk, pamanīju, ka arī trešā šķirne parādās kā mikroskopiski punkti starp ziliem un, ko es nevaru paskaidrot, arī starp sarkaniem punktiem.

```
plot_mapper_final(map_res3, "Mapper (n = 7, overlap = 85%)")
```

Mapper (n = 7, overlap = 85%)



Palielinot intervālu skaitu līdz 7, attēlā parādās visas trīs vīna šķirnes. Šķirnēm atbilstošo virsotņu izvietojums atbilst tam, kas bija redzams pirmo divu galveno komponentu vizualizācijā. Trešā vīna šķirne, kurai ir mazāks novērojumu skaits, ir reprezentēta tikai ar vienu virsotni.

Faktoru analīze

Faktoru analīzē tiek mēģināts izskaidrot datu savstarpējās korelācijas ar latentajiem faktoriem, kurus pēc tam var interpretēt.

Veicu faktoru analīzi ar diviem faktoriem, izmantojot Varimax rotāciju, kas pagriež faktoru asis tā, lai katrs mainīgais būtu spēcīgi saistīts ar vienu faktoru un vāji ar otru, padarot rezultātus vieglāk interpretējamus.

```
fa_res <- factanal(vins_numeric, factors = 2, rotation = "varimax", scores = "regression")
```

Apskatu faktoru svarus:

```
print(fa_res$loadings, cutoff = 0.3)

##
## Loadings:
##               Factor1 Factor2
## Alcohol                0.726
## Malic_Acid            -0.475
## Ash                   0.324
## Alcalinity           -0.354
## Magnesium             0.352
## Total_Phenols         0.829  0.340
## Flavanoids           0.927
## Nonflavanoid_Phenols -0.549
## Proanthocyanins       0.619
## Color_Intensity      -0.412  0.815
## Hue                   0.663
## OD280_OD315          0.868
## Proline               0.391  0.615
##
##               Factor1 Factor2
## SS loadings    4.123   2.166
## Proportion Var 0.317   0.167
## Cumulative Var 0.317   0.484
```

Abi faktori kopumā izskaidro aptuveni 48,4% no datu variācijas.

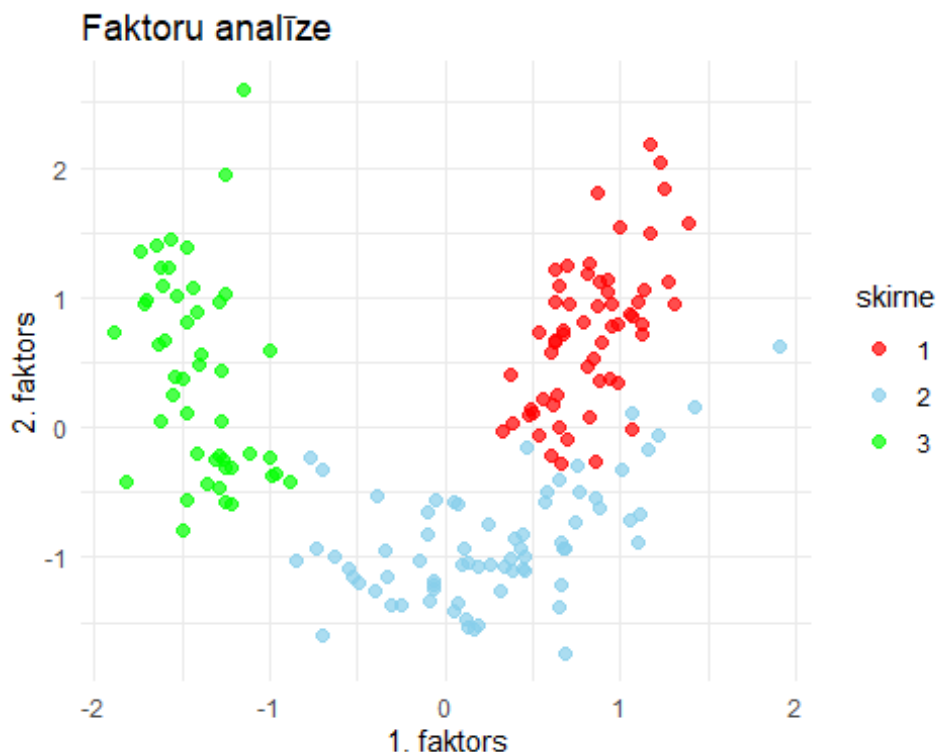
Pirmais faktors izskaidro 31,7% no variācijas un to galvenokārt veido šādi mainīgie: Flavanoids (0,927), OD280_OD315 (0,868), Total_Phenols (0,829), Hue (0,663), Proanthocyanins (0,619), Alcohol (0,726), Proline (0,391), Malic_Acid (-0,475), Nonflavanoid_Phenols (-0,549) un Alcalinity (-0,354). Kopumā šķiet, ka pirmais faktors atspoguļo vīnu ķīmisko “raksturu”.

Otrais faktors izskaidro 16,7% no variācijas un to veido salīdzinoši maz mainīgo, galvenokārt: Color_Intensity (0,815), Proline (0,615), Total_Phenols (0,340) u.c. Visticamāk, šis faktors atspoguļo vīnu krāsu intensitāti un ar to saistītos rādītājus.

Vizuālizēju faktorus:

```
fa_scores <- as.data.frame(fa_res$scores)
fa_scores$skirne <- vins_labels # Pievienojam atpakaļ klases krāsām

ggplot(fa_scores, aes(x = Factor1, y = Factor2, color = skirne)) +
  geom_point(size = 2, alpha = 0.7) +
  scale_color_manual(values = class_cols) +
  theme_minimal() +
  labs(title = "Faktoru analīze",
       x = "1. faktors", y = "2. faktors")
```



Ar faktoru analīzes pieeju iegūtais attēls ir gandrīz identisks tam, kas tika iegūts, vizualizējot pirmās divas galvenās komponentes. Tas notika tāpēc, ka faktoru analīzē tika izmantots iestatījums `scores = "regression"`, ar kuru tiek aprēķinātas punktu koordinātas kā lineāras kombinācijas no oriģinālajiem mainīgajiem, kas ir ļoti līdzīgas tam, ko iegūst PCA analīzē.

Secinājums: Ar vienkāršiem datiem faktoru analīze un galveno komponentu analīze dod ļoti līdzīgus rezultātus, jo abas metodes meklē lineāras kombinācijas, kas atspoguļo galvenos datu gradientus.

Izmantotā literatūra

Madukpe, V.N., Ugoala, B.C., Zulkepli, N.F.S., 2025. A Comprehensive Review of the Mapper Algorithm, a Topological Data Analysis Technique, and Its Applications Across Various Fields (2007-2025). <https://doi.org/10.48550/arXiv.2504.09042>